Introduction to Machine Learning - 9 -

Principal component analysis (PCA)

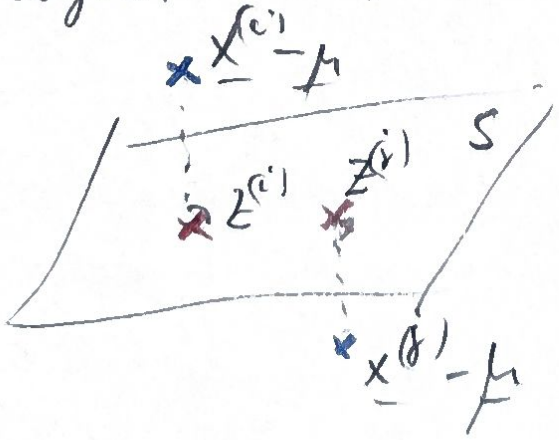
Idea: reduce dimension of data
- save memory / training time
- reduce overfitting

Linear: PCA: project onto the eigenvectors of the largest eigenvalues of the covariance matrix

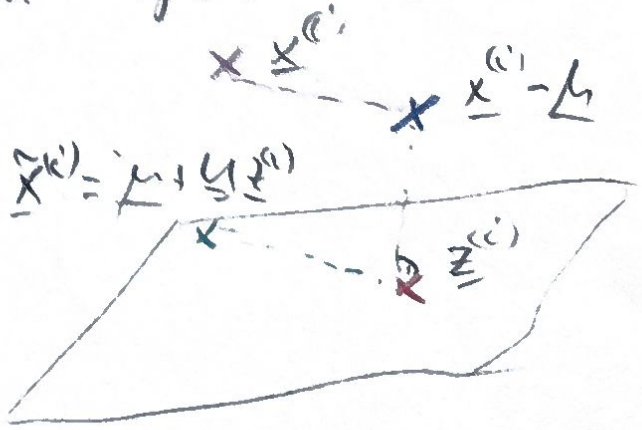
Nonlinear: auto encoders, use a neural network to project the data to a subspace

Original data: D dim, $\underline{\mu}$ mean, $\underline{\mu} = \frac{1}{N}\sum_i \underline{x}^{(i)}$

Projection to a K dim. subspace S



$\underline{U}$: orthogonal base of S in D, it is also a projection operator:



$\underline{z}^{(i)} = \underline{U}^T (\underline{x}^{(i)} - \underline{\mu})$

↑ representation of $\underline{x}^{(i)}$,

$\tilde{\underline{x}}^{(i)} = \underline{\mu} + \underline{U}\underline{z}^{(i)}$ Reconstruction

Reconstruction error:

$$\frac{1}{N}\sum_{i=1}^{N} \| \underline{x}^{(i)} - \underline{\tilde{x}}^{(i)} \|^2 \quad \text{to be minimized}$$

The subspace S is useful if the data is scattered which allows separation

Second criteria: $\sum_{j=1}^{K} \mathrm{var}(z_j) = \max$

↖ variance in each direction

$$\sum_{j=1}^{K} \mathrm{var}(z_j) = \frac{1}{N}\sum_{j=1}^{K}\sum_{i=1}^{N} (z_j^{(i)} - \bar{z}_j)^2 = \frac{1}{N}\sum_{j=1}^{K}\sum_{i=1}^{N} \| z_j^{(i)} - \vec{z}_j \|^2 =$$

$$= \frac{1}{N}\sum_{i} \| z^{(i)} \|^2 \quad \text{since} \quad \underline{\bar{z}} = 0$$

↑ we removed the mean from $\underline{x}$ before projection

We show that the two criteria are equivalent!

Observation (1): $\| \underline{\tilde{x}}^{(i)} - \underline{\mu} \| = \| \underline{U} \underline{z}^{(i)} \| = \| \underline{z}^{(i)} \|$

Observation (2): if A is to be minimized and B is to be maximized at the same time and $A + B = \text{const}$, then A and B are equivalent

IML - 9 - 01/11

$$\frac{1}{N}\sum_{i=1}^{N}\|\underline{x}^{(i)}-\underline{\tilde{x}}^{(i)}\|^2+\frac{1}{N}\sum_{i=1}^{N}\|\underline{z}^{(i)}\|^2 =$$

$$=\frac{1}{N}\sum_{i=1}^{N}\|\underline{x}^{(i)}-\underline{\tilde{x}}^{(i)}\|^2+\frac{1}{N}\sum_{i=1}^{N}\|\underline{\tilde{x}}^{(i)}-\underline{\mu}\|^2 \quad (*)$$

$\underline{x}^{(i)}-\underline{\tilde{x}}^{(i)}$: perpendicular to S

$\underline{\tilde{x}}^{(i)}-\underline{\mu}$: inside S

Pithagoras!

$$(*)\ \frac{1}{N}\sum_{i=1}^{N}\|\underline{x}^{(i)}-\underline{\tilde{x}}^{(i)}+\underline{\tilde{x}}^{(i)}-\underline{\mu}\|^2=\frac{1}{N}\sum_{i=1}^{N}\|\underline{x}^{(i)}-\underline{\mu}\|^2=\text{const}$$

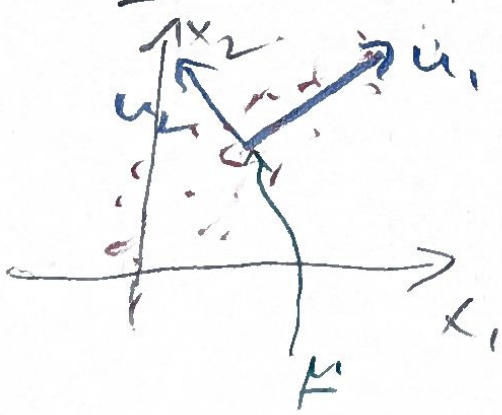
independent of S

QED

outer (dyadic) prod

Covariance matrix

$$\underline{\underline{\Sigma}}=\frac{1}{N}\sum_{i=1}^{N}(\underline{x}^{(i)}-\underline{\mu})(\underline{x}^{(i)}-\underline{\mu})^T=\frac{1}{N}\sum_{i=1}^{D}(\underline{x}^{(i)}-\underline{\mu})\circ(\underline{x}^{(i)}-\underline{\mu})$$



it is almost like a moment of inertia, the magnitude of an eigenvalue indicates the importance of that direction

Eigenvectors associated with the largest eigenvalues are called principal components.

Let us take $K=1$ as an example

$\underline{u}$ is the unit vector in this case $\underline{u} \in S$

z is scalar $z^{(i)} = \underline{u}^T (\underline{x}^{(i)} - \underline{\mu})$

$$\frac{1}{N} \sum_i \left\{ z^{(i)} \right\}^2 = \frac{1}{N} \sum_i \left[\underline{u}^T (\underline{x}^{(i)} - \underline{\mu}) \right]^2 =$$

$$= \frac{1}{N} \sum_i \underline{u}^T (\underline{x}^{(i)} - \underline{\mu}) (\underline{x}^{(i)} - \underline{\mu}) \underline{u} =$$

$$= \underline{u}^T \underbrace{\left[\frac{1}{N} \sum_i (\underline{x}^{(i)} - \underline{\mu}) (\underline{x}^{(i)} - \underline{\mu}) \right]}_{\underline{\underline{\Sigma}}} \underline{u} = \underline{u}^T \underline{\underline{\Sigma}} \underline{u} =$$

$$= \underline{u}^T \underline{\underline{Q}} \underline{\underline{\Lambda}} \underline{\underline{Q}}^T \underline{u} \quad (*)$$

Spectral decomposition:

$\underline{\underline{Q}}$: columns are eigenvectors (q_j)

$\underline{\underline{\Lambda}}$: diagonal with eigenvalues (λ_j)

$\underline{a} = \underline{\underline{Q}}^T \underline{u} \rightarrow \underline{u} = \underline{\underline{Q}} \underline{a}$

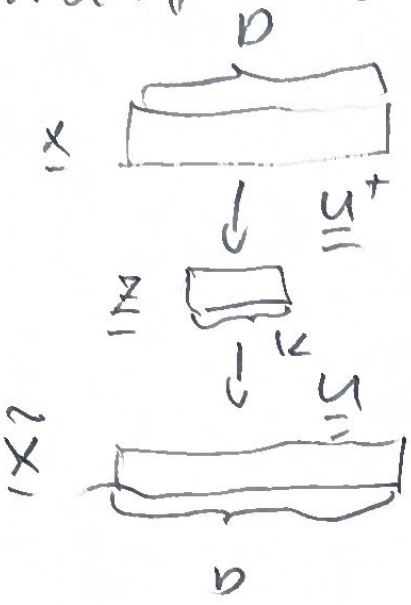
$$(*) \quad \underline{a}^T \underline{\underline{\Lambda}} \underline{a} = \sum_{j=1}^{D} \lambda_j a_j^2$$

$\|a\| = 1$ because $\underline{u}$ is unitary and $\underline{u}$ is unit vector

Largest if $a_1 = 1$ $a_j = 0$ $j > 1$, with the general assumption that eigenvalues are ordered

Autoencoders:

What we did:

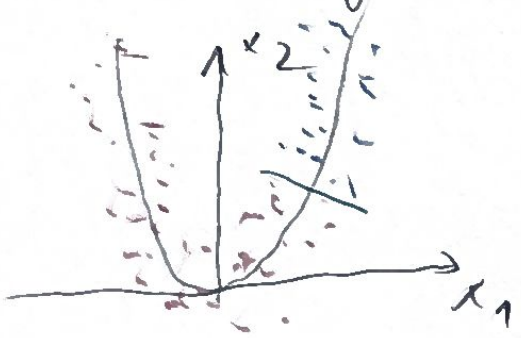


Teaching input, target: same

$L = \|x - \tilde{x}\|^2$

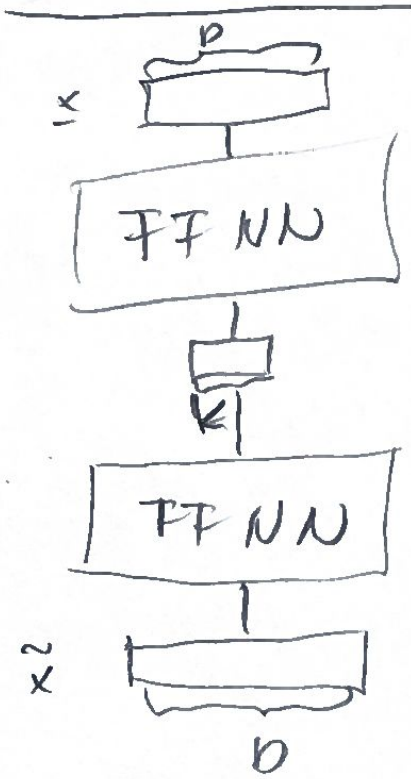
So in principle we could have used a neural network to do PCA.

What if linear encoders do not work.



Would be best to project to the parabola

Nonlinear autoencoders



Teaching: same input and output

Autoencoders: unsupervised learning

- noise reduction
- data compression
- anomaly detection
- generative modeling